

Computer Science & IT

Data Structure & Programming



Tree

Lecture No. 05



By- Abhishek Sir

Recap of Previous Lecture



Topic

construction of Binary tree with traversal

Topic

pre - post tree construction (2-ary)

Topic

only pre and only post (BST)

Topic

BST construction and comparison

Topic

Topics to be Covered



Topic

Search, min, max, delete

Topic

Height balanced tree (AVL tree)

Topic

Topic

Topic



Topic : Search



```
int Search(Tnode *t, int k) {
```

```
    if (t == NULL)
```

```
        return -1;
```

```
    else if (t->data == k)
```

```
        return 1;
```

```
    else if (t->data < k)
```

```
        return Search(t->right, k);
```

```
    else
```

```
        return Search(t->left, k);
```

```
}
```

Time Complexity

depends upon Height

$$O(h)$$



Topic : Search



Height

$$\lfloor \log_2 n \rfloor \leq h < n-1$$

Min

Max



Time complexity

depends upon Height

$$O(h)$$



Topic : BST



Min - element - : May be a Nonleaf



```
int Min(Tnode *t) {
```

```
if(t) {
```

```
while (t->left != null)
```

```
t = t->left
```

```
return t->data;
```

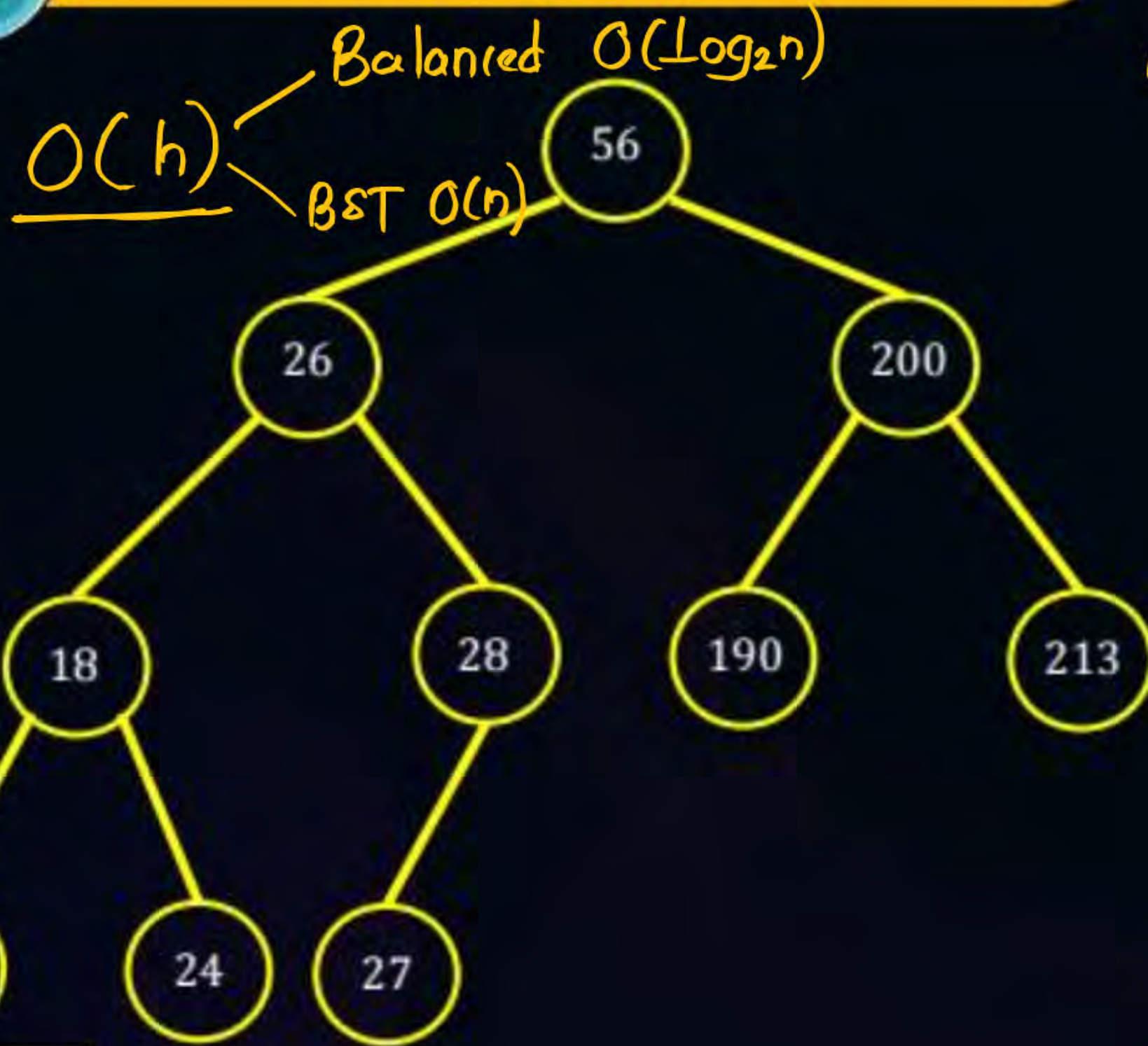
```
}
```

```
return -1;
```

```
}
```



Topic : BST



```
int max(Tnode *t) {  
    if (!t) {  
        while (t->right != NULL)  
            t = t->right;  
        return t->data;  
    }  
    return -1;  
}
```



Topic : Tree



#Q. Consider the following program that takes a tree root & an integer element as an argument

```
int count (struct node * node, int element) {  
    if (node == null)  
        return 0;  
    else {  
        int result = 0;  
        if (node -> data == element)  
            result = 1;  
        return result + count(node-> left, element)  
            + count (node -> right, element);  
    }  
}
```



if root node of above
tree with element = 2
the value return by fun is
2



Slide

```

graph TD
    2((2)) --> 3L((3))
    2 --> 1((1))
    3L --> 3LL((3))
    1 --> 2L((2))
    1 --> 2R((2))

```

$$C(3)$$

$$0 + C(\text{Null}) + C(\text{Null}) = 1$$

Diagram illustrating the recursive calculation of the number of ways to climb 3 steps (Count(3)).

The diagram shows a tree structure representing the recursive calls:

- Root node: $\text{Count}(3)$ (labeled 3 above it).
- Level 1 nodes: $\text{Count}(2)$ (labeled 2 above it) and $\text{Count}(1)$ (labeled 1 above it).
- Level 2 nodes (from $\text{Count}(2)$): $\text{Count}(1)$ (labeled 1 above it) and $\text{Count}(0)$ (labeled 0 above it).
- Level 2 nodes (from $\text{Count}(1)$): $\text{Count}(0)$ (labeled 0 above it) and $\text{Count}(-1)$ (labeled -1 above it).
- Level 3 nodes (from $\text{Count}(0)$): $\text{Count}(-1)$ (labeled -1 above it) and $\text{Count}(-2)$ (labeled -2 above it).

The diagram illustrates the recursive process of calculating the number of ways to climb 3 steps, showing the sequence of recursive calls and the base cases.



Topic : Question

A Binary Search Tree (BST) stores values in the range 37 to 573. Consider the following sequence of keys.

I. 81, 537, 102, 439, 285, 376, 305 X

II. 52, 97, 121, 195, 242, 381, 472

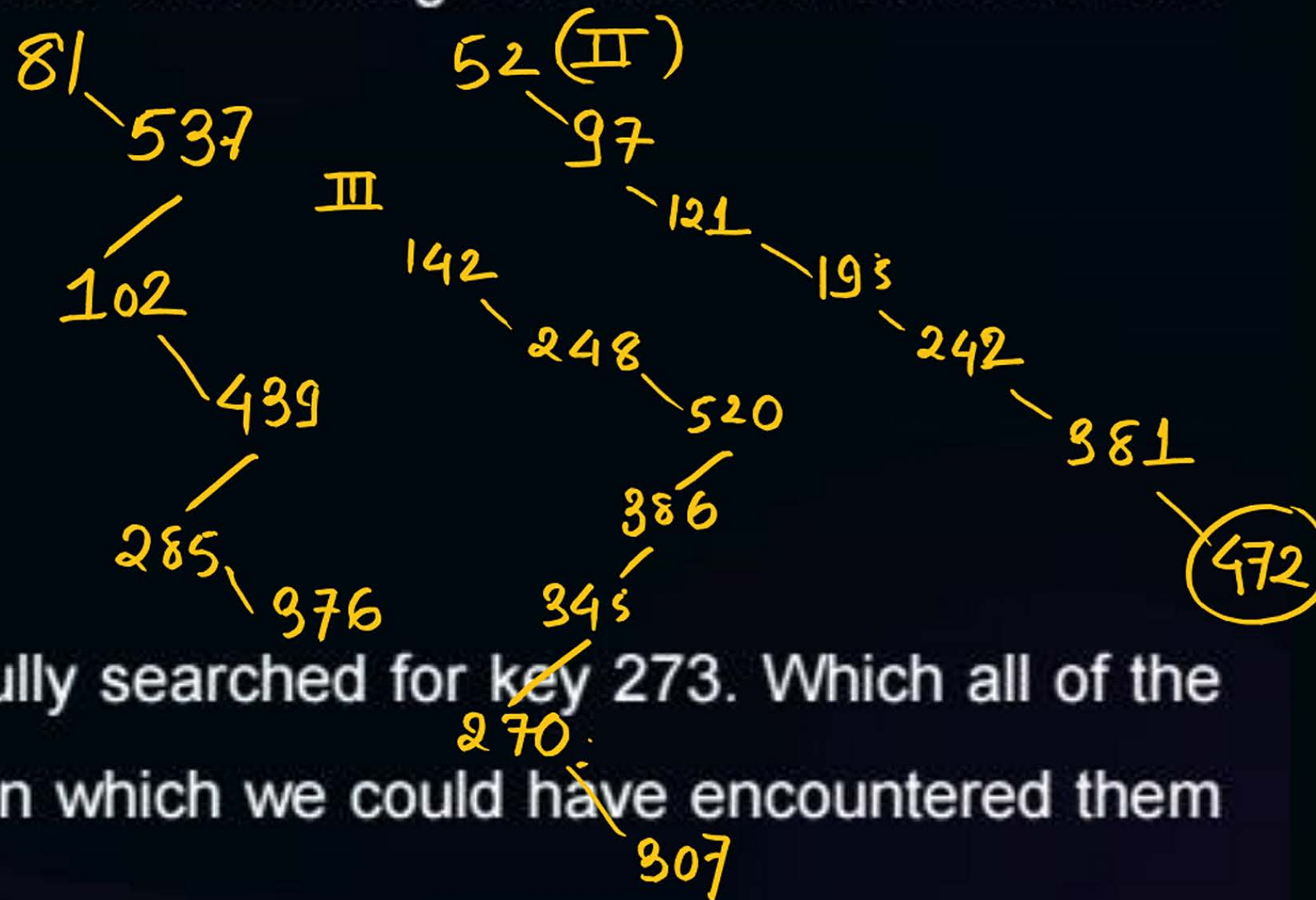
III. 142, 248, 520, 386, 345, 270, 307 ✓

IV. 550, 149, 507, 395, 463, 402, 270

Suppose the BST has been unsuccessfully searched for key 273. Which all of the above sequence list nodes in the order in which we could have encountered them in the search

(A) II and III only

(b) I and III only (c) III and IV only (d) III only





Topic : Question

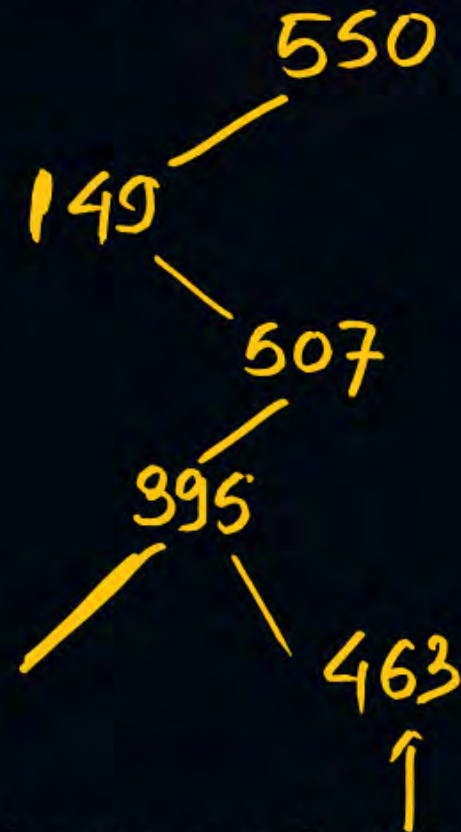
A Binary Search Tree (BST) stores values in the range 37 to 573. Consider the following sequence of keys.

I. 81, 537, 102, 439, 285, 376, 305

II. 52, 97, 121, 195, 242, 381, 472

III. 142, 248, 520, 386, 345, 270, 307

IV. 550, 149, 507, 395, 463, 402, 270



Suppose the BST has been unsuccessfully searched for key 273. Which all of the above sequence list nodes in the order in which we could have encountered them in the search

(A) II and III only

(b) I and III only (c) III and IV only (d) III only ✓

Question

#Q suppose BST is formed using sequential representation of array consists of 37 elements. The first element is store index 0.

What is the index of

(A) Smallest element : $w = 31$

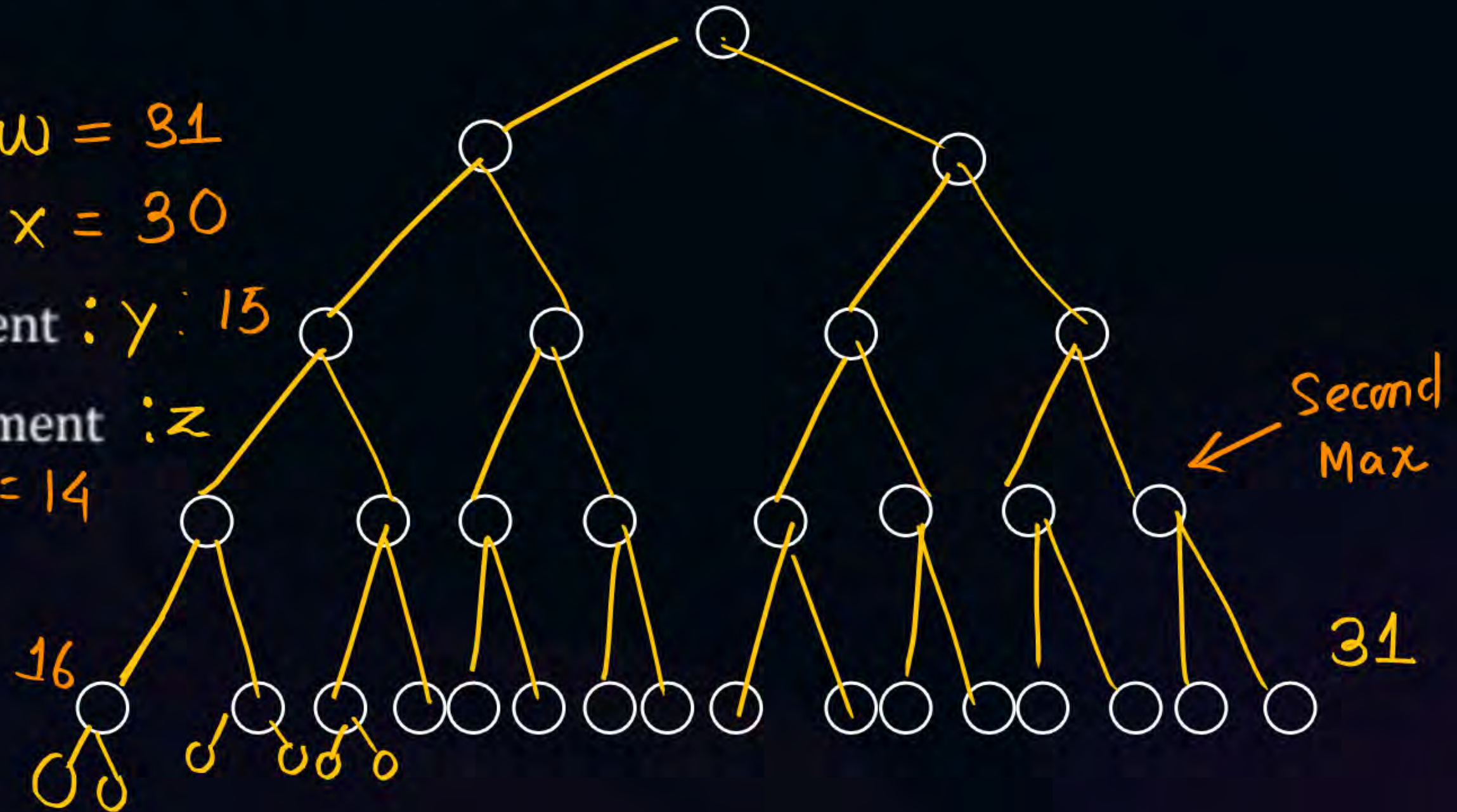
(B) Maximum element : $x = 30$

(C) Second smallest element : $y = 15$

(D) Second Maximum element : z

$$w + x + y + z$$

$$= 14$$





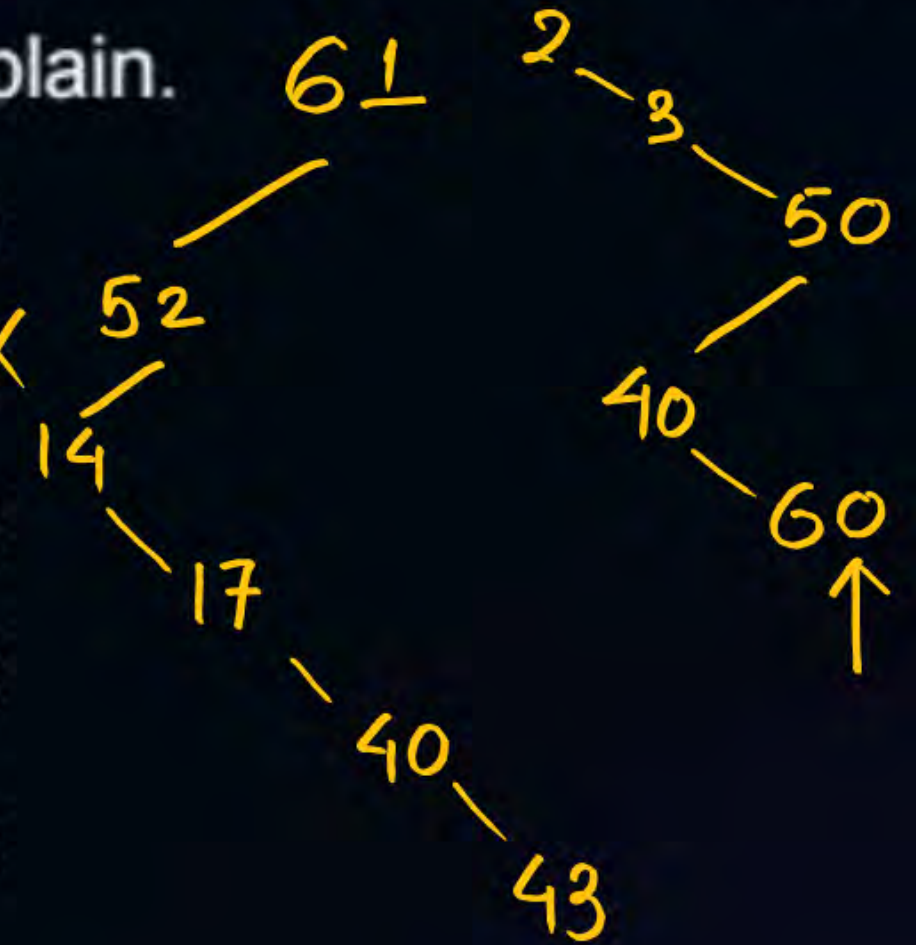
Topic : Question



Answer - No possible

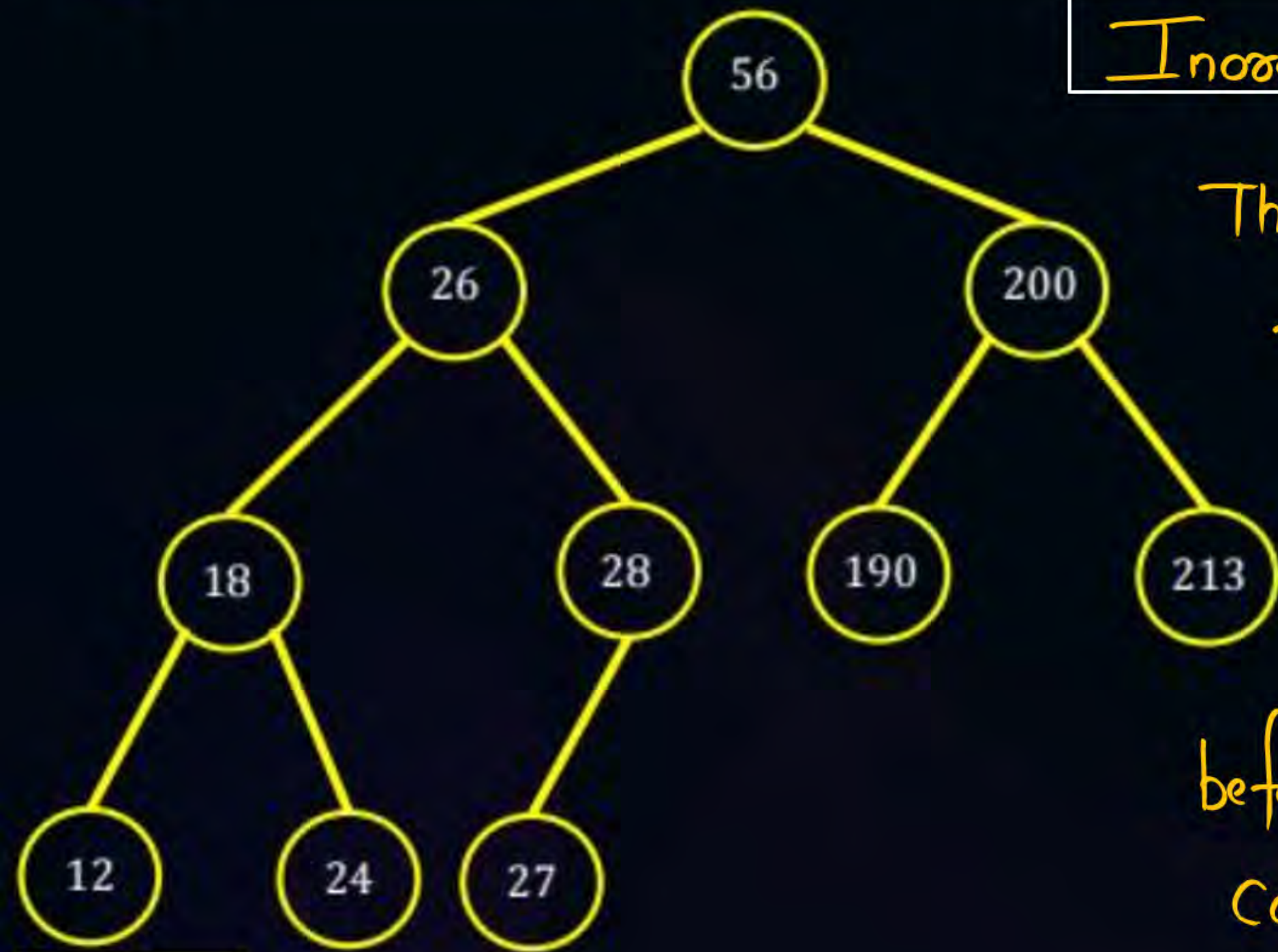
Q A binary search tree is used to locate the number 43. Which of the following probe sequences are possible and which are not? Explain.

(a) 61	52	14	17	40	✓	43
(b) 2	3	50	40	60	✗	43
(c) 10	65	31	48	37		43
(d) 31	61	52	14	41		43
(e) 17	77	27	66	18		43





Topic : BST



Inorder Successor (Max element successor, NULL)

The node comes after x in inorder

traversal is called Inorder Successor of x

Inorder predecessor

(Min predecessor, NULL)

The node that comes before node x in Inorder traversal is called Inorder predecessor of x

BST Deletion

Delete node x from a tree T

We can distinguish three cases

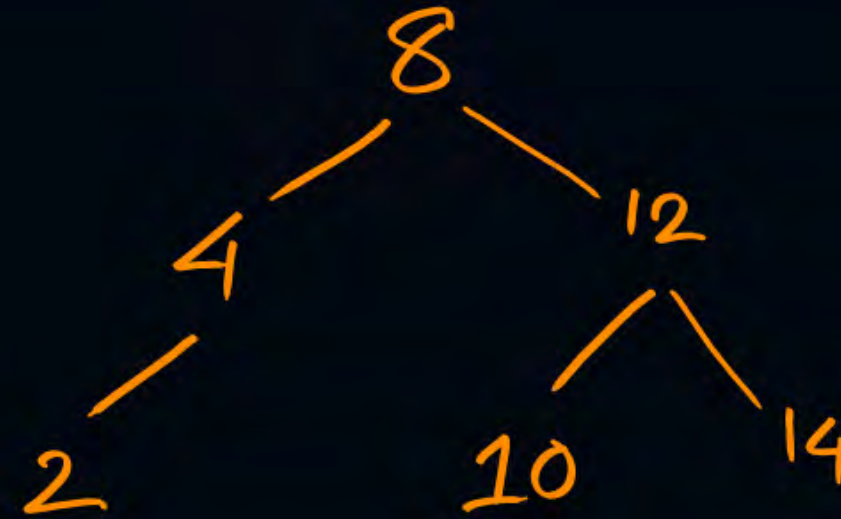
x has *no* children

x has *one* child

x has *two* children

Deletion Case 1

If x has no children – just remove x

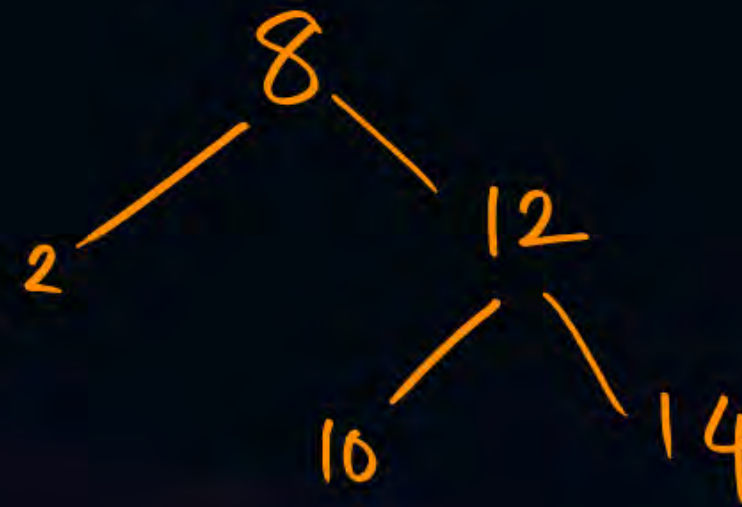


Deletion Case 2

If x has exactly one child



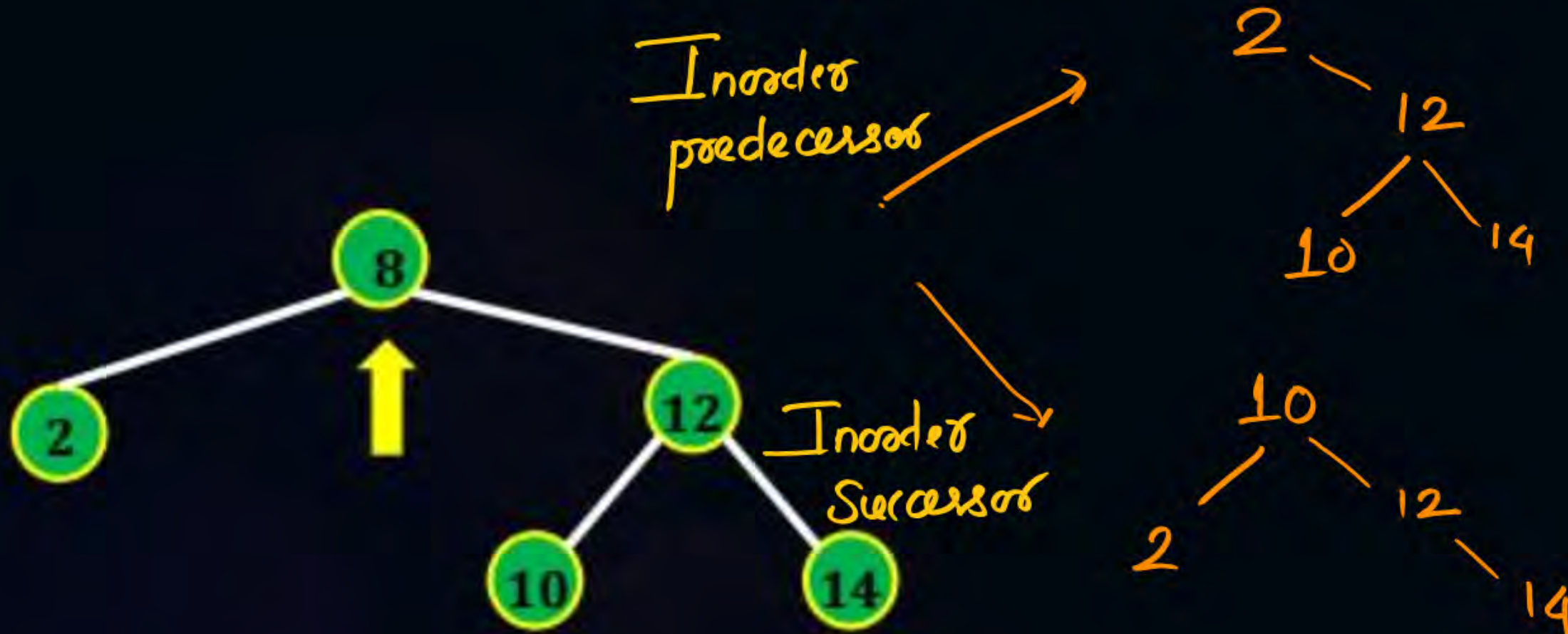
if x is having 1 children then
then children of x becomes children of
parent of (x)



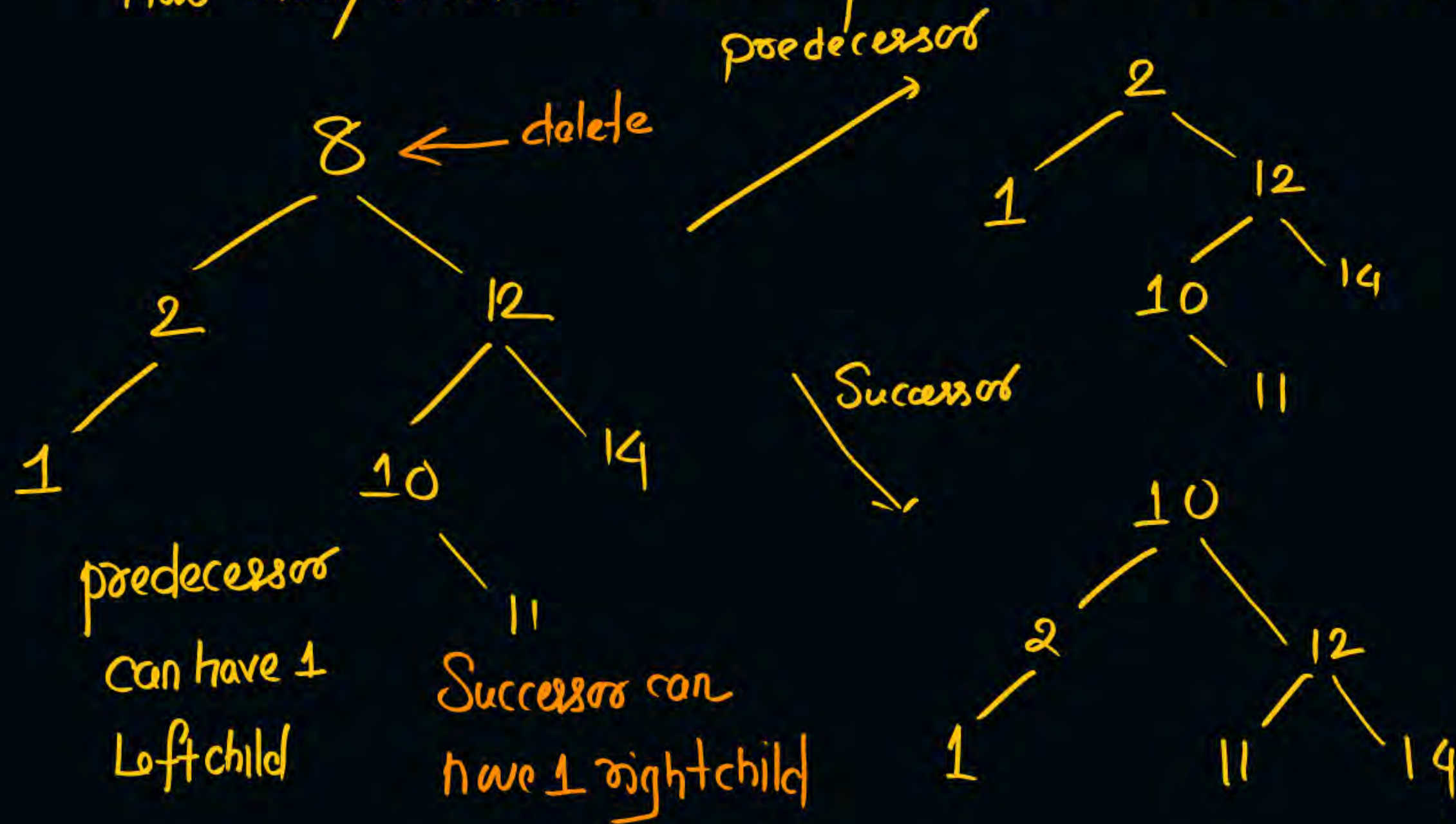
Deletion Case 3

If x has two children, then to delete it we have to

replace by inorder Successor or Inorder predecessor



How many children Inorder predecessor or successor can have — 1





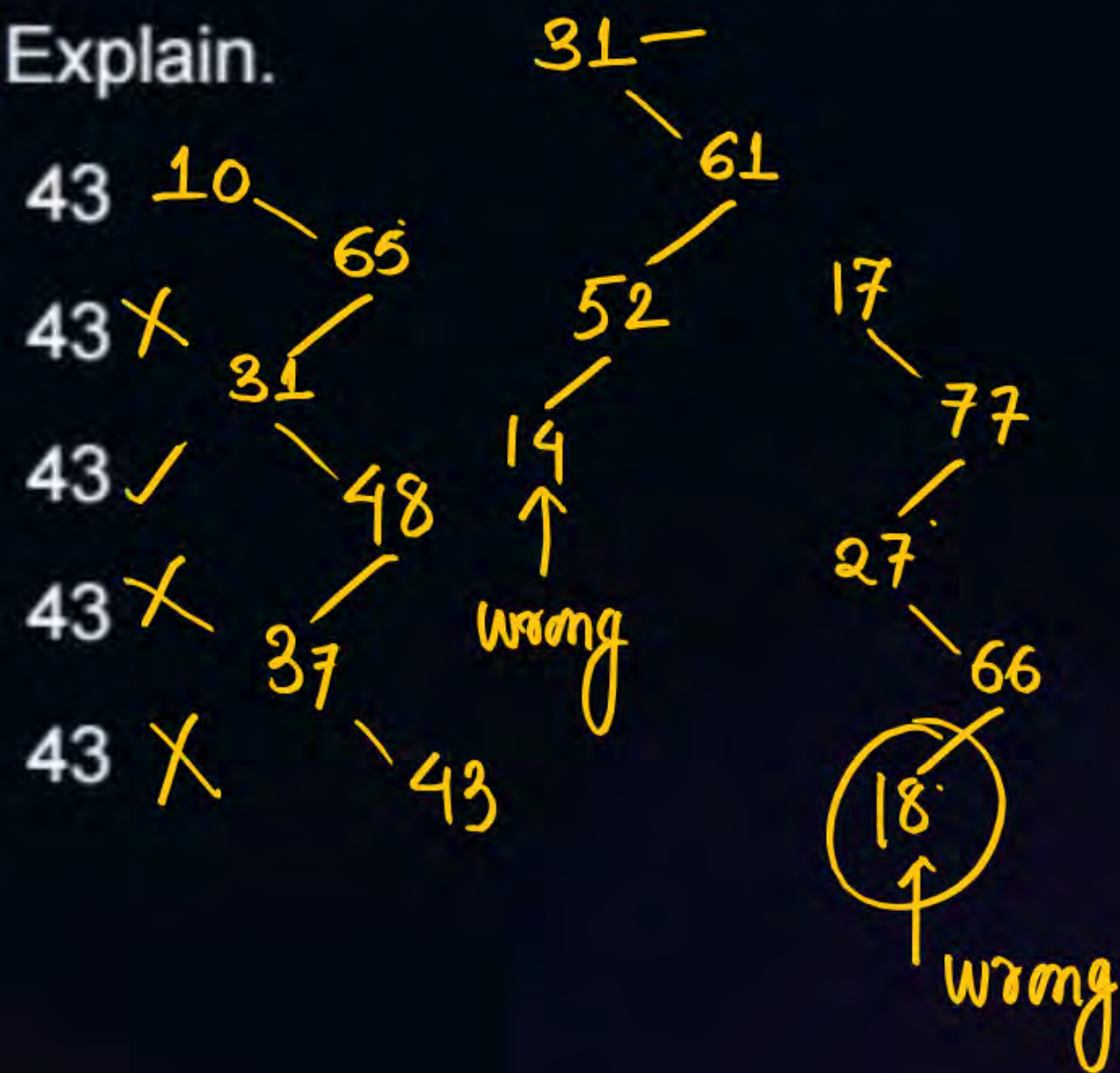
Topic : Question



Answer — No possible

Q A binary search tree is used to locate the number 43. Which of the following probe sequences are possible and which are not? Explain.

- | | | | | |
|--------|----|----|----|----|
| (a) 61 | 52 | 14 | 17 | 40 |
| (b) 2 | 3 | 50 | 40 | 60 |
| (c) 10 | 65 | 31 | 48 | 37 |
| (d) 31 | 61 | 52 | 14 | 41 |
| (e) 17 | 77 | 27 | 66 | 18 |





Topic : Question



Consider the function and the binary tree shown.

```
int foo(node *p) {  
    int retval;  
    if (p == NULL)  
        return 0;  
    else {  
        retval = p->val + foo(p->left) + foo(p->right);  
        printf("%d ", retval);  
        return retval;  
    }  
}
```

```
typedef struct node {  
    int val;  
    struct node *left, *right;  
} node;
```



Topic : Question

Consider the function and the binary tree shown.

```
int foo(node *p) {  
    int retval;  
    if (p == NULL)  
        return 0;  
    else {  
        retval = p->val + foo(p->left) + foo(p->right);  
        printf("%d ", retval);  
        return retval;  
    }  
}
```



what is the output

3, 8, 16, 13, 24

- A. 3 8 5 13 11 10
- B. 3 5 8 10 11 13
- C. 3 8 16 13 24 50
- D. 3 16 8 50 24 13



Topic : Question



Consider the function and the binary tree shown.

```
int foo(node *p) {  
    int retval;  
    if (p == NULL)  
        return 0;  
    else {  
        retval = p->val + foo(p->left) + foo(p->right);  
        printf("%d ", retval);  
        return retval;  
    }  
}
```





Question



40, 20, 10, 30, 60, 50, 70

Assume an initially empty BST and following function is executed

```
struct node* foo(struct node* root, int value) {  
    if (root == NULL)  
        return newNode(value);  
    if (value < root->val)  
        root->left = foo(root->left, value);  
    else if (value > root->val)  
        root->right = foo(root->right, value);  
    return root;  
}
```

```
int main() {  
    struct node* root = NULL;  
    int arr[] = {40, 20, 10, 30, 60, 50, 70};  
    for (int i = 0; i < 7; i++) {  
        root = foo(root, arr[i]);  
    }  
}
```

What will be the pre-order traversal of the final BST?

- A. 40, 20, 10, 30, 60, 50, 70 ✓
- B. 40, 20, 10, 30, 50, 60, 70
- C. 40, 10, 20, 30, 60, 50, 70
- D. 40, 20, 30, 10, 60, 70, 50



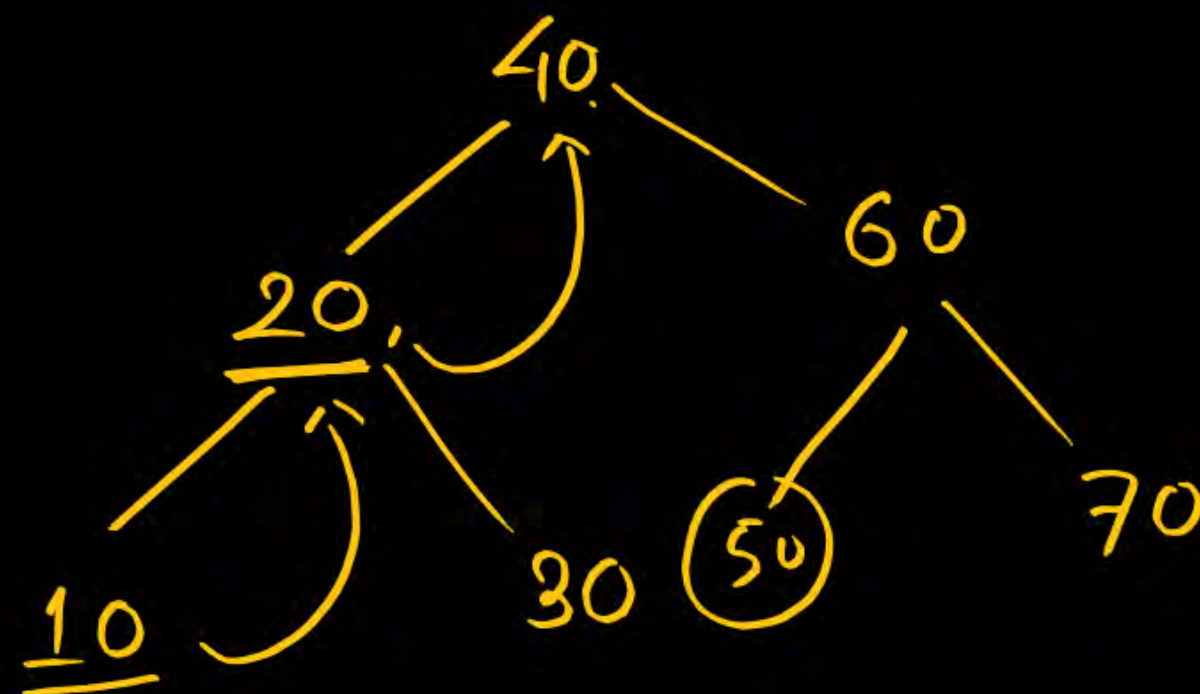
Question



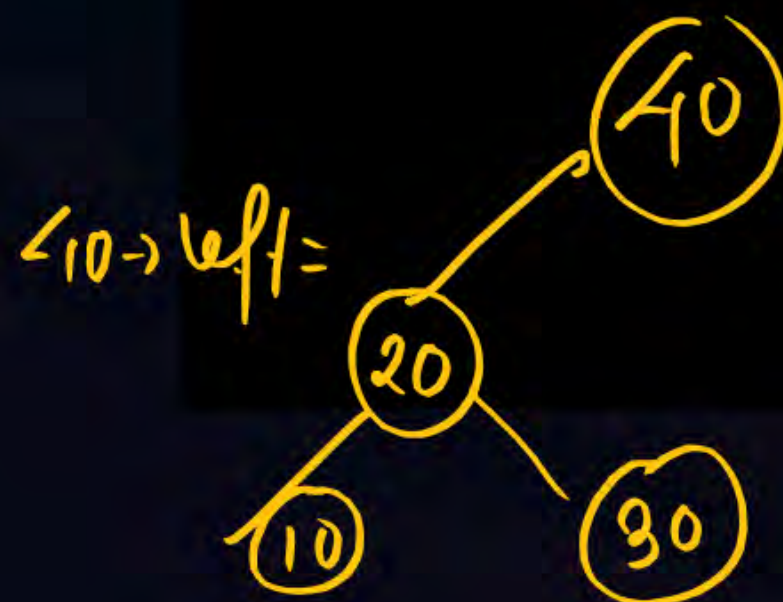
Assume an initially empty BST and following function is executed

```
struct node* foo(struct node* root, int value) {  
    if (root == NULL)  
        return newNode(value);  
    if (value < root->val)  
        root->left = foo(root->left, value);  
    else if (value > root->val)  
        root->right = foo(root->right, value);  
    return root;  
}
```

40, 20, 10, 30, 60, 50, 70



40, 20, 10, 30



THANK - YOU